HYPNEXIS'26 · TRACK 01 · AI-01

Kavach

Catching AI-cloned voices before the money moves

Project document: problem, solution, system design, data, evaluation, prototype, testing and demo plan.

Problem statement: AI-01 Voice-Clone Detection for Phone Scams

Event: HYPNEXIS'26, 24–25 October 2026

Round 1 submission: PPT, prototype and presentation by 3:00 AM, 25 October

Contents

1. Executive summary 4
2. The problem 5
2.1 Problem statement (as given) 5
2.2 Why it matters 5
2.3 Key challenges and our answer to each 5
3. Our solution 6
3.1 The core insight 6
3.2 Why a frozen pretrained model 6
3.3 Removing shortcuts by design 6
3.4 Three-way verdict 6
4. System architecture 7
4.1 Components 7
4.2 Request flow 7
4.3 API contract 7
5. Data 8
5.1 Datasets 8
5.2 Why not ASVspoof 2019 8
5.3 Splits 8
5.4 Ethics and consent 8
6. Evaluation 9
6.1 Metric: Equal Error Rate (EER) 9
6.2 Experiments 9
7. Prototype and user experience 10
7.1 Design direction: minimal neumorphism 10
7.2 Screens and states 10
8. System design considerations 11
8.1 Latency budget (target, single clip of 10 s) 11
8.2 Scaling path 11
8.3 Security and privacy design 11
9. Testing plan 12
10. Demo plan 13
11. Risks and limitations 14
12. Impact and roadmap 15
12.1 Who would use it 15
12.2 Next steps 15
13. Build timeline (24–25 October) 16

14. References 17
15. Results 18
15.1 Phase 1 model (In-the-Wild only) 18
15.2 Final model (Phase 2): every test condition 18
15.3 Phase 2 requirements 18
15.4 Our own cloners (never in training) 19
15.5 Load test 19
15.7 Red-team test: zero-shot clone from a WhatsApp note 19
15.6 Known limits 20

1. Executive summary

Scammers can now copy anyone's voice from a few seconds of audio taken from a reel, a WhatsApp voice note or a short call. They then phone a parent or a bank customer in that voice and ask for money. People trust a familiar voice, so the scam works.

Kavach is a detector that listens to a 2–10 second speech clip and returns one of three verdicts: **REAL**, **NOT SURE** or **FAKE (AI clone)**, together with a confidence score and a timeline showing which seconds looked synthetic.

The central challenge is generalisation. A detector trained on known cloning tools tends to learn their specific quirks and fails on new ones. Detectors trained on public benchmarks also learn shortcuts such as silence length or loudness, which do not exist in real calls.

Our approach rests on one idea: every cloning tool, old or new, must finish by turning its internal representation into a waveform through a vocoder or neural codec. That step leaves fine fingerprints. We detect those fingerprints with XLS-R, a speech model pretrained on 436,000 hours of real speech in 128 languages, kept frozen, with a small classifier on top. We remove shortcuts by design (silence trimming, loudness normalisation, phone-line simulation) and prove generalisation by testing on speakers and cloning tools the model never saw in training.

The prototype is a minimal, neumorphic web app: upload a clip or record from the microphone, and see the verdict, score and suspicious-seconds timeline. A FastAPI backend serves the model. The whole system is covered by unit, integration, end-to-end, security and load tests.

2. The problem

2.1 Problem statement (as given)

Scammers can clone the voice of a relative or bank officer from just a few seconds of audio. Build a detector that decides whether a 2–10 second speech clip is genuine or synthetic, and keeps working on voice-cloning tools never seen during training.

Pretrained self-supervised speech models are allowed. The ASVspoof 2019 evaluation set and the In-the-Wild dataset are suggested for testing generalisation.

2.2 Why it matters

- Voice cloning has become cheap, fast and open-source. A convincing clone needs only seconds of reference audio.
- The victims are often elderly parents, who are least able to verify a caller by other means.
- The money usually moves through instant payments, so there is little time to reverse it.
- Phones and messaging apps today give no signal that a voice might be synthetic.

2.3 Key challenges and our answer to each

Challenge (from the PS)	Why it is hard	Kavach's answer
Generalising to unseen voice generators	Models memorise the artifacts of the tools in their training set	Frozen speech model that knows real speech deeply; tested on 4 generators never used in training
Short clips with limited evidence	2 seconds carries little information	Sliding-window scoring with aggregation; an explicit NOT SURE zone instead of guessing
Different languages and speakers	Training data is mostly English	XLS-R base pretrained on 128 languages; our own Hindi / Gujarati / English test set; train and test speakers never overlap
Misleading shortcuts (silence, loudness)	Benchmarks leak labels through silence length and level	Silence trimming and loudness normalisation on every clip; a shortcut test that proves the model does not use them

3. Our solution

3.1 The core insight

Every cloner ends in a vocoder. A cloning system first decides what to say and in whose voice, then a vocoder or neural codec renders that into sound. Vocoders are good enough to fool ears but leave consistent traces: unnatural fine structure in high frequencies, overly smooth micro-variation, phase patterns that real vocal tracts do not produce, and missing natural micro-noise. New cloners change the front end often, but the rendering step is shared by all of them.

3.2 Why a frozen pretrained model

XLS-R has heard a vast amount of real human speech. Its internal layers already encode what real speech looks like, at many levels from raw acoustics to phonetics. We read all 24 of its transformer layers, learn which layers carry the strongest fake-detection signal, and train only a small classifier on top.

- **Generalisation:** a frozen model cannot overfit its representation to one cloner's artifacts.
- **Feasibility:** fits a 6 GB laptop GPU; features are computed once and cached, so each retraining takes minutes.
- **Multilingual:** pretraining covered 128 languages, including Hindi and Gujarati.

3.3 Removing shortcuts by design

1. **Silence trimming:** leading and trailing silence removed with an energy threshold, so silence length carries no information.
2. **Loudness normalisation:** every clip scaled to the same RMS level, so loudness carries no information.
3. **Phone-line simulation:** random down-sampling to 8 kHz and back, band-pass to the telephone band, light noise, so the model is robust to call audio.
4. **Shortcut test:** a classifier trained only on silence duration and loudness is measured on the same data. If it performs well, the data leaks; we report it and show that our model's score does not depend on these features.

3.4 Three-way verdict

Verdict	Meaning	What the user sees
REAL	Score below the lower threshold	Green. "This sounds like a real person."
NOT SURE	Score between the thresholds, or clip too short after trimming	Amber. "Can't be sure. Verify another way."
FAKE	Score above the upper threshold	Red. "Likely AI-cloned. Hang up and call back on a saved number."

Thresholds are set on a held-out calibration split, not on test data.

4. System architecture

4.1 Components

Component	Technology	Responsibility
Web app	Plain HTML/CSS/JS, minimal neumorphic UI	Upload or record audio, show verdict, score, timeline, history of this session
API server	Python, FastAPI, Uvicorn	Validate input, decode audio, run the detector, return JSON
Audio front end	ffmpeg, NumPy	Decode any common format to 16 kHz mono, trim silence, normalise loudness
Feature extractor	XLS-R 300M (frozen), PyTorch, CUDA when available	Hidden states of all layers, pooled per window
Classifier	Learned layer weights + logistic regression / small MLP	Score each window; aggregate to a clip score
Calibration	Thresholds from a calibration split	Map score to REAL / NOT SURE / FAKE
Training pipeline	Python scripts	Index data, extract and cache features, train, evaluate, export model

4.2 Request flow

```
Browser ──(multipart audio, ≤ 10 MB)──► POST /api/analyze
   1. size and type check           reject early, 413 / 415
   2. ffmpeg decode → 16 kHz mono   reject undecodable, 422
   3. trim silence, normalise
   4. windows of 4 s, hop 1 s       (one window if clip shorter)
   5. XLS-R layers → pooled vectors
   6. classifier → window scores → clip score
   7. thresholds → verdict
◄── JSON { verdict, score, duration, windows[], model, latency_ms }
   audio buffer discarded; nothing written to disk
```

4.3 API contract

Endpoint	Input	Output
GET /api/health	none	{ status, model_loaded, device }
POST /api/analyze	multipart field "file": wav, flac, mp3, m4a, ogg, webm; max 10 MB; 1–30 s	{ verdict, score (0–1), confidence, duration_s, speech_s, windows: [{start, end, score}], thresholds, latency_ms }
GET /	none	the web app

5. Data

5.1 Datasets

Dataset	Content	Role
In-the-Wild (Müller et al., Interspeech 2022)	Real and deepfake speech of 58 public figures collected online, 37.9 h, English, Apache-2.0	Main training and testing, split by speaker so test speakers are never heard in training
Kavach-Indic (ours)	Team members reading Hindi, Gujarati and English sentences, recorded on phones, with consent	Real side of our own test set
Kavach-Clones (ours)	Clones and synthetic speech made with local tools: Kokoro TTS, Veena Hindi TTS, SNAC neural-codec re-synthesis, RVC voice conversion	Fake side of our own test set; every generator is unseen in training

5.2 Why not ASVspoof 2019

ASVspoof 2019 LA was the first choice, but its host server delivered 53 KB/s at the venue, which would take about 38 hours for 7 GB. In-the-Wild is the harder, real-world benchmark the PS also names, and our own modern cloners add generators released after 2019.

5.3 Splits

- **Speaker-disjoint:** speakers are assigned to train (≈70%), calibration (≈10%) and test (≈20%). No speaker appears in two splits.
- **Unseen generators:** the detector is trained only on In-the-Wild. Every one of our own generators (SNAC, Kokoro, Veena, RVC) is used only for testing, so each is a cloner the model has never seen.
- **Clip length:** evaluation clips are cut to 2–10 s to match the PS.

5.4 Ethics and consent

- Every voice we clone belongs to a team member who agreed to it.
- Clones are used only for testing and the demo, and are deleted after the event.
- The API keeps audio in memory only and writes nothing to disk.

6. Evaluation

6.1 Metric: Equal Error Rate (EER)

Set the decision threshold so that the share of real voices wrongly called fake equals the share of fake voices wrongly called real. That shared percentage is the EER. Lower is better: 0% is perfect, 50% is a coin toss. We also report accuracy of the three-way verdict and the share of NOT SURE answers.

6.2 Experiments

#	Experiment	Question it answers
E1	Baseline on unseen speakers (In-the-Wild test split)	Does it work on people it never heard?
E2	Shortcut test: silence + loudness only classifier	Does the data leak labels? Does our model rely on it?
E3	Clip length 2 s / 4 s / 10 s	How much audio does it need?
E4	Phone-quality audio (8 kHz telephone band)	Does it survive a phone line?
E5	Each of our 4 generators, never seen in training	Does it catch a cloner it never saw?
E6	Hindi / Gujarati subset	Does it work across languages?

Reporting rule: every number in the presentation is produced by a script in the repository and stored in runs/. No number is typed in by hand. Weak results are shown with their reason.

7. Prototype and user experience

7.1 Design direction: minimal neumorphism

- A single soft surface colour; controls appear pressed into or raised from it using paired light and dark shadows.
- One accent colour per verdict: green REAL, amber NOT SURE, red FAKE. Nothing else competes for attention.
- Large central verdict dial, readable from across a room during the demo.
- Neumorphism has known contrast problems, so all text keeps a contrast ratio of at least 4.5:1 and focus states are clearly visible.
- Light and dark themes, phone and desktop layouts.

7.2 Screens and states

State	What the user sees
Idle	Title, short line on what Kavach does, two raised buttons: Upload clip, Record. Drop zone.
Recording	Record button pressed in, live level meter, timer counting to 10 s, Stop.
Analysing	Dial gently pulses; "Listening for clone fingerprints…"
Result	Dial fills to the score in the verdict colour, verdict word, confidence, advice line, waveform timeline with suspicious seconds shaded.
Error	Plain message saying what went wrong and how to fix it (too long, unsupported format, no speech found).
History	Small list of this session's checks with verdict chips; cleared on reload.

8. System design considerations

8.1 Latency budget (target, single clip of 10 s)

Stage	Target
Upload + decode	< 300 ms
Pre-processing	< 50 ms
XLS-R features (GPU)	< 400 ms
Classifier + verdict	< 10 ms
Total server time	< 1 s

Measured values are reported by the load test; these are targets, not results.

8.2 Scaling path

- The API is stateless: any number of replicas behind a load balancer.
- Model loads once per process; requests share it under a lock or a small batching queue.
- CPU-only deployment possible by exporting to ONNX and using a smaller backbone for low-cost tiers.
- For telecom-scale use, scoring moves to streaming windows with a running verdict.

8.3 Security and privacy design

Threat	Control
Huge uploads / resource exhaustion	10 MB request cap, 30 s duration cap, decode timeout, one inference at a time per worker
Malformed or malicious media files	Decoding in a separate ffmpeg process with fixed arguments, no shell, timeout; output format forced to raw PCM
Path traversal / filename injection	Uploaded filename never used on disk; audio handled as bytes in memory
Cross-site abuse	CORS limited to the app origin; no cookies or sessions
Data leakage	No audio stored, no audio in logs; logs carry only duration, verdict and latency
Dependency vulnerabilities	pip-audit on requirements; Semgrep and Bandit static analysis
Misuse of the verdict	NOT SURE zone; UI advice says to verify by calling back, never to trust the score alone

9. Testing plan

Layer	What we test	Tool
Unit	Silence trim, normalisation, windowing, score aggregation, threshold mapping, EER function	pytest
Model sanity	Known real clip scores lower than known fake clip; output is deterministic for the same input	pytest
API integration	Health endpoint, valid upload, each error code (413, 415, 422), JSON schema of the response	pytest + FastAPI TestClient
End to end	Browser opens app, uploads a clip, sees a verdict; record flow with a fake microphone stream	Playwright
Security	Oversized file, wrong type, corrupted file, zero-length file, file with a path-like name; static analysis; dependency audit	pytest, Semgrep, Bandit, pip-audit
Load / system	Latency p50 / p95 per request, behaviour under concurrent requests, memory stays flat across many requests	Python load script
Accessibility	Contrast, keyboard navigation, focus visibility, labels	Manual + automated checks

Pass criteria: all tests green, no high-severity static-analysis findings, no known-vulnerable dependencies, p95 latency within budget on the demo laptop.

10. Demo plan

1. **Hook:** "Scammers need just a few seconds of your voice." Play a teammate's real recording. Kavach shows green REAL.
2. **The scam:** play the same teammate's cloned voice saying "Mummy, I'm in trouble, send ₹50,000".
3. **Caught:** Kavach turns red FAKE, suspicious seconds highlighted.
4. **Unseen tool:** play a clone from a generator the model never trained on. Still red.
5. **Live:** a judge speaks into the microphone. Green REAL.
6. **Proof:** results slide: unseen speakers, unseen generator, Hindi, phone audio.

A screen recording of the full demo is kept as a backup in case anything fails live.

11. Risks and limitations

Risk / limitation	Mitigation
No detector is perfect, especially on very short clips	NOT SURE zone; we report errors honestly
In-the-Wild is English only	Our own Hindi and Gujarati test set
Our own clone set is small	Reported as a stress test, not as a benchmark
Heavy noise or deliberate attacks can lower accuracy	Phone-line simulation helps; adversarial robustness is future work
Venue internet	Datasets and model weights downloaded early; demo runs fully offline

12. Impact and roadmap

12.1 Who would use it

- **Banks and call centres:** flag cloned voices impersonating customers.
- **Telecom operators:** network-level warning on suspicious calls.
- **Families:** a phone app, or a small Bluetooth guard device for parents, since phone apps cannot read call audio directly.

12.2 Next steps

- Live call mode with streaming windows.
- Larger multilingual training set covering more Indian languages.
- Combining voice score with scam-language detection (urgency, money requests).
- Partial fine-tuning of the top XLS-R layers on a larger GPU.

13. Build timeline (24–25 October)

Time	Phase	Output
15:00–16:00	P0 Setup	Environment, model weights, dataset indexed
16:00–17:30	P1 Baseline	Cached features, first classifier, first EER
18:15–20:15	P2 Generalisation	Shortcut test, phone test, own clones, held-out generator table
21:30–23:30	P3 Prototype	API + neumorphic web app working end to end
23:30–01:00	P4 Testing + numbers	All test suites green, results into slides
01:00–02:45	P5 Rehearsal + submit	Two timed rehearsals, backup video, submission

14. References

- Müller, N. et al. (2022). Does Audio Deepfake Detection Generalize? Interspeech 2022. (In-the-Wild dataset)
- Müller, N. et al. (2021). Speech is Silver, Silence is Golden: What do ASVspoof-trained Models Really Learn? ASVspoof 2021 Workshop.
- Babu, A. et al. (2022). XLS-R: Self-supervised Cross-lingual Speech Representation Learning at Scale. Interspeech 2022.
- Wang, X. & Yamagishi, J. (2023). Spoofed training data for speech spoofing countermeasure can be efficiently created using neural vocoders. ICASSP 2023.
- Wang, X. et al. (2020). ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech. Computer Speech & Language.

15. Results

Every number below is read from the result files produced by the scripts in this repository (runs/.json). Nothing is typed by hand.*

15.1 Phase 1 model (In-the-Wild only)

Test (unseen speakers)	EER	n
E1 unseen speakers 4s	2.31%	4026
E3 len 2s	2.56%	4026
E3 len 10s	2.26%	4026
E4 phone line	2.85%	4026

Shortcut-only classifier (duration, silence, loudness): EER 47.00%. Near 50% means the data does not leak the label through silence or loudness.

15.2 Final model (Phase 2): every test condition

Trained on In-the-Wild (clean, phone-line, channel-augmented, replayed, noisy), modern neural TTS voices and clean real speech (LibriSpeech). About 25% of the modern TTS voices and all test speakers are never seen in training.

Condition	EER	Fakes flagged	False alarms on real
itw:clean	2.01%	97.10%	0.17%
itw:phone	2.96%	94.96%	0.98%
itw:len2	2.26%	96.80%	0.26%
itw:len10	2.01%	97.27%	0.17%
libri_n:noisetr	–	–	0.00%
modern:clean	0.50%	98.04%	0.16%
modern:replay	10.26%	93.14%	18.89%
libri:clean	–	–	0.00%
libri:replay	–	–	18.89%
libri:chan	–	–	3.11%
itw_p2test:clean	1.80%	97.30%	0.20%
itw_p2test:r8k	1.40%	97.30%	0.10%
itw_p2test:amr	1.80%	97.10%	0.50%
itw_p2test:opus	2.10%	97.70%	1.00%
itw_p2test:noise	3.50%	87.99%	0.60%
itw_p2test:chan	11.11%	80.58%	4.80%

15.3 Phase 2 requirements

Requirement	Result	Target	Status
R1 channel robustness	worst noise: 1.94× clean (clean 1.80%, r8k 1.40%, amr 1.80%, opus 2.10%, noise 3.50%, chan 11.11%)	≤ 2.5×	Met
R2 partial-fake localisation	F1 80.48% (P 79.95%, R 81.02%) on 100 partial fakes	report F1 at 0.5 s	Reported
R3 streaming	update every 500 ms; p50 25.7 ms, p95 31.4 ms per update (cuda)	500 ms updates	Met
R4 Phase 1 guard	unseen speakers 4s -0.30 pts; len 2s -0.30 pts; len 10s -0.25 pts; phone line +0.10 pts	within 2 pts	Met

15.4 Our own cloners (never in training)

Generator	n	EER vs real	Phone-line EER	Flagged FAKE
KOKORO	46	0.06%	8.69%	100.00%
RVC	97	1.03%	1.03%	95.88%
SNAC	300	8.67%	9.01%	42.00%

15.5 Load test

Sequential: median 318 ms, p95 728 ms per request. Concurrent (4 workers, 60 requests): median 1218 ms, 0 errors, 3.4 requests/s.

15.7 Red-team test: zero-shot clone from a WhatsApp note

A speaker was cloned with F5-TTS, a zero-shot cloner never used in training, from a 3.9-second WhatsApp voice note. Three high-quality clones (NFE 64, denoised reference) were then laundered six ways each: as-is, WhatsApp Opus, phone AMR-NB, street noise 15 dB, speaker-to-mic replay, Opus + noise.

	FAKE	NOT SURE	REAL (missed)
Clones, before noise shield	12	0	6
Clones, with noise shield	16	2	0
Real voice note, with noise shield	0	1	5

Finding: background noise hid the clone fingerprint. The noise shield fixes most of this: when the estimated SNR is below 24 dB, the clip is also scored after Demucs vocal isolation. A red FAKE needs a denoised score above 0.90; a weaker signal gives NOT SURE.

Held-out set (shield on, through the app)	Result
Noisy fakes, 15 dB (n=100)	FAKE 76.00%, NOT SURE 10.00%, missed 14.00% (without shield: caught 54.00%)
Noisy real voices, 15 dB (n=100)	REAL 89.00%, NOT SURE 9.00%, wrongly FAKE 2.00%
Clean real voices (n=100)	REAL 98.00%, wrongly FAKE 1.00%

The 0.90 cut-off and 24 dB switch were chosen on the 200 noisy validation clips listed in runs/denoise_validation.json; the clean-voice row uses different clips. The Phase 2 R1 numbers in 15.3 are measured without the shield.

15.6 Known limits

- Audio played through a speaker into a microphone (replay) is the hardest case: accuracy drops and some real replayed voices are flagged. The NOT SURE zone and the advice to call back exist for this reason.
- Commercial assistants such as GPT and Gemini voice were not available as training data; modern neural TTS voices were used as the closest open substitute.
- In-the-Wild is English; Hindi coverage comes from Kokoro and Veena voices and team recordings.